

國立中央大學機械工程學系

OM5003 自動化光學檢測

Automatic Optical Inspection

期末考報告

撰寫人：陳泓任

授課教師：李朱育 教授

民國 114 年 6 月 11 日

目錄

第一章 題目敘述.....	2
第二章 使用方法.....	3
第三章 輸出結果解析.....	3
第四章 錯誤處理與優化說明.....	9

1. 題目一使用方法

首先將圖一切割成 12 塊拼圖碎片並依照編號命名儲存以作為樣板資料，操作程式碼與切分結果如圖二所示。



圖一、拼圖完成圖以各片拼圖編號

```
import cv2
import os

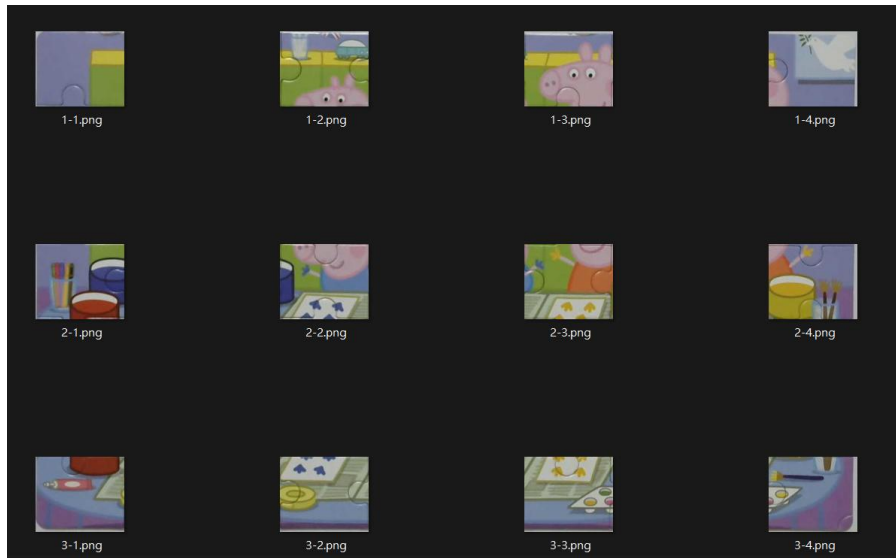
# 讀取圖片
image = cv2.imread(r"final\prob_1\init_image.png")
height, width, _ = image.shape

# 切割參數
rows = 3
cols = 4
tile_height = height // rows
tile_width = width // cols

# 輸出資料夾
output_dir = "final/prob_1/train_set"
os.makedirs(output_dir, exist_ok=True)

# 切割與儲存
for row in range(rows):
    for col in range(cols):
        y1 = row * tile_height
        y2 = (row + 1) * tile_height
        x1 = col * tile_width
        x2 = (col + 1) * tile_width
        tile = image[y1:y2, x1:x2]
        filename = f"{row+1}-{col+1}.png"
        cv2.imwrite(os.path.join(output_dir, filename), tile)
```

(a) 切分拼圖之程式碼



(b) 切分結果

緊接著設定 `load_templates` 函數讀取儲存的樣板資料，如圖二(b)所示，並建立 ORB 特徵點 `kp` 與計算描述子 `des`，最後將結果儲存在字典 `template` 中，程式碼如圖三所示。

```
def load_templates(template_dir):
    '''載入樣板拼圖碎片並建立 ORB 特徵點和描述子'''
    templates = {}
    orb = cv2.ORB_create()
    for fname in os.listdir(template_dir):
        if fname.endswith('.png'):
            label = fname[:-4] # e.g., '1-1'
            img = cv2.imread(os.path.join(template_dir, fname), cv2.IMREAD_GRAYSCALE)
            kp, des = orb.detectAndCompute(img, None)
            templates[label] = (img, kp, des)
    return templates
```

圖三、讀取樣板 ORB 資訊

擷取並儲存到 ORB 資訊後，設定 `match_piece_to_templates` 函數透過 `detect_puzzle_pieces` 將影像中的一塊碎片，匯入並轉成灰階以建立 ORB 特徵點 `kp2` 與計算描述子 `des2`。

然後將 `des2` 與 `des1` 進行 ORB 描述子比對，以找出相似度最高的那一塊，程式碼如圖四所示。其中 `des1` 使用 `rotate_image` 將樣本資料進行每 90 度的旋轉，以提高 ORB 特徵比對的準確率，程式碼如圖五所示。

```

def match_piece_to_templates(piece_img, templates: dict, matcher: cv2.BFMatcher, orb):
    piece_gray = cv2.cvtColor(piece_img, cv2.COLOR_BGR2GRAY)
    kp2, des2 = orb.detectAndCompute(piece_gray, None)

    best_label = None
    max_good_matches = 0

    for label, (tpl_img, kp1, des1) in templates.items():
        if des1 is None or des2 is None:
            continue

        # 建立四種旋轉角度的樣板圖像與特徵
        for angle in [0, 90, 180, 270]:
            rotated = rotate_image(tpl_img, angle)
            kp_rot, des_rot = orb.detectAndCompute(rotated, None)
            if des_rot is None:
                continue

            matches = matcher.knnMatch(des_rot, des2, k=2)
            good = [m for match in matches if len(match) == 2 for m, n in match if m.distance < 0.75 * n.distance]

            if len(good) > max_good_matches:
                max_good_matches = len(good)
                best_label = label

    return best_label, max_good_matches

```

圖四、拼圖比對

```

def rotate_image(image, angle):
    h, w = image.shape[:2]
    center = (w//2, h//2)
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    rotated = cv2.warpAffine(image, M, (w, h), flags=cv2.INTER_LINEAR)
    return rotated

```

圖五、旋轉圖像

為獲取影像中的一塊碎片以進行比對，設定 detect_puzzle_pieces 函數進行檢測影像中的所有拼圖碎片，程式碼如圖六所示，其中包含對影像進行灰階、模糊化、取邊界、膨脹與侵蝕以及找輪廓，其中過濾掉輪廓面積小於 1000 以及大於 10000 的部分。同時使用 cv.BFMatcher() 創建 matcher，以利於在 match_piece_to_templates 中進行比對與標記。

當找到輪廓時，首先利用 extract_aligned_piece 尋找其最小外接矩並將擷取到的圖像旋轉為正擺的狀態以利於提高 ORB 特徵比對的準確率，最後再呼叫 match_piece_to_templates 函數進行比對以及繪製外框與編號，程式碼如圖七所示。

```

def detect_puzzle_pieces(img, template_dir):
    orb = cv2.ORB_create()
    matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)

    templates = load_templates(template_dir)
    original = img.copy()

    # 把圖轉為灰階並模糊去噪
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # 用Canny偵測邊緣
    edges = cv2.Canny(blurred, 40, 80)

    # 膨脹邊緣讓輪廓封閉
    kernel = np.ones((5,5), np.uint8)
    dilated = cv2.dilate(edges, kernel, iterations=1)
    eroded = cv2.erode(dilated, kernel, iterations=1)

    # 找輪廓
    contours, _ = cv2.findContours(eroded, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 1000 or area > 10000: # 設定合理的大小範圍
            continue
        x, y, w, h = cv2.boundingRect(cnt)
        piece = extract_aligned_piece(img, cnt)

        label, match_count = match_piece_to_templates(piece, templates, matcher, orb)
        if label:
            rect = cv2.minAreaRect(cnt)
            box = np.intp(cv2.boxPoints(rect))
            cv2.drawContours(original, [box], 0, (0,255,0), 2)
            cv2.putText(original, label, (x+w/2, y+h/2), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
    return original

```

圖六、檢測拼圖碎片

```

def extract_aligned_piece(img, contour):
    rect = cv2.minAreaRect(contour)
    box = cv2.boxPoints(rect)
    box = np.intp(box)

    width = int(rect[1][0])
    height = int(rect[1][1])

    src_pts = box.astype("float32")
    # 計算對齊矩形的目標點位置
    dst_pts = np.array([[0, height-1],
                        [0, 0],
                        [width-1, 0],
                        [width-1, height-1]], dtype="float32")

    # 取得仿射變換矩陣並校正圖像方向
    M = cv2.getPerspectiveTransform(src_pts, dst_pts)
    warped = cv2.warpPerspective(img, M, (width, height))

    return warped

```

圖七、正擺拼圖碎片

最後設定 main 函數引入影片並呼叫 detect_puzzle_pieces 進行辨識，再將結果輸出於 Result 介面上，程式碼如圖八所示。

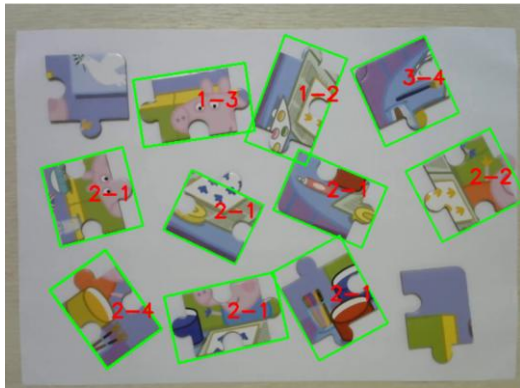
```
def main():
    cap = cv2.VideoCapture(r'final\prob_1\puzzle.mp4')
    ret, frame = cap.read()
    while ret:
        result = detect_puzzle_pieces(frame, "final/prob_1/train_set/manual")
        cv2.imshow("Result", result)
        ret, frame = cap.read()
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

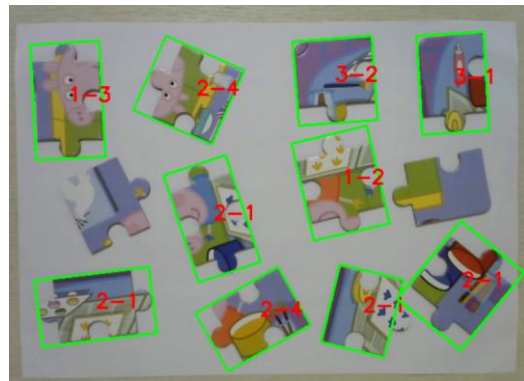
圖八、主程式

2. 題目一輸出結果解析

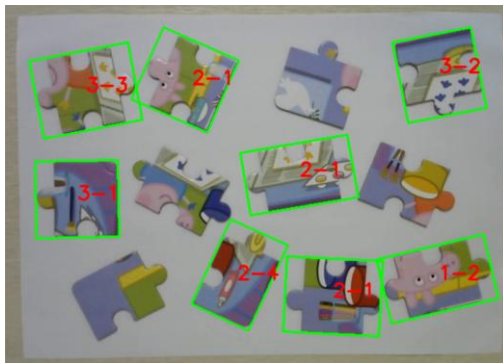
輸出結果如圖九所示，可以得知 1-1 以及 1-4 的拼圖特徵與所有拼圖碎片相似度最低，無法被辨識到；而 2-1 的拼圖特徵與所有拼圖碎片相似度最高，容易在同一張影像中辨識到多個 2-2 拼圖。



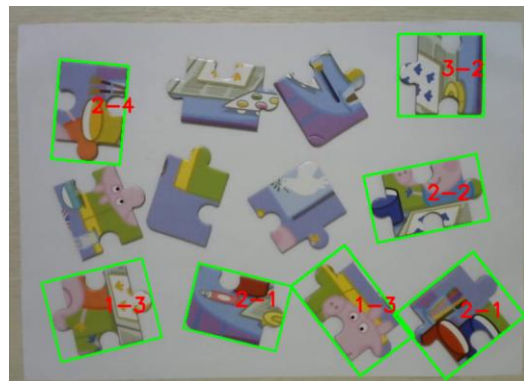
(a) 第一張



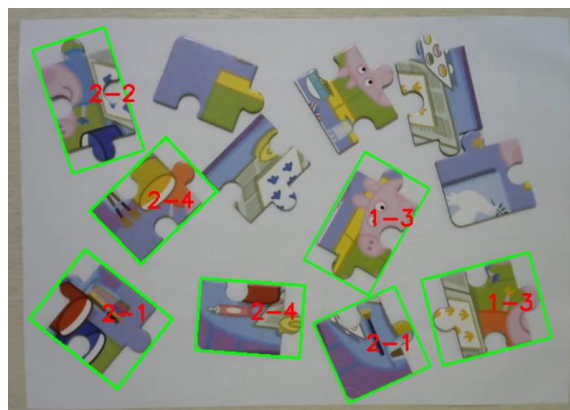
(b) 第二張



(c) 第三張



(d) 第四張



(e) 第五張

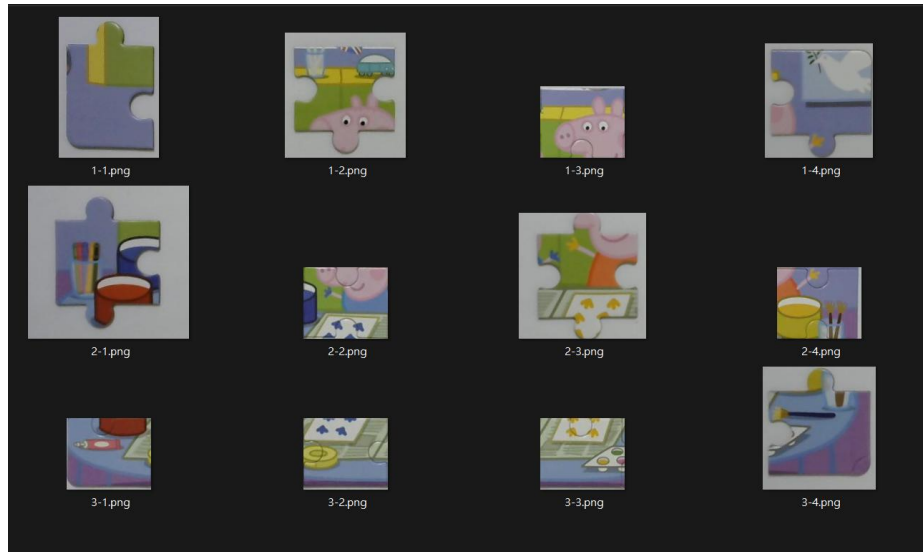
圖九、辨識結果

3. 題目一錯誤處理與優化說明

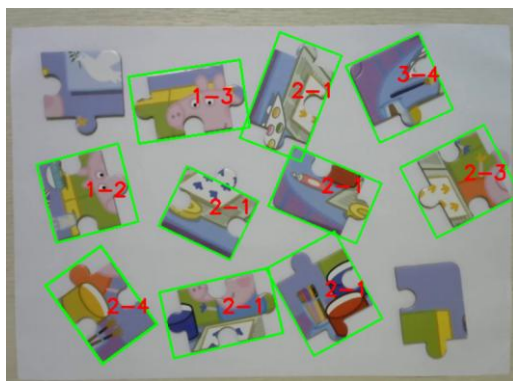
為提高辨識效率，進行以下操作：

- 更換樣板檔案

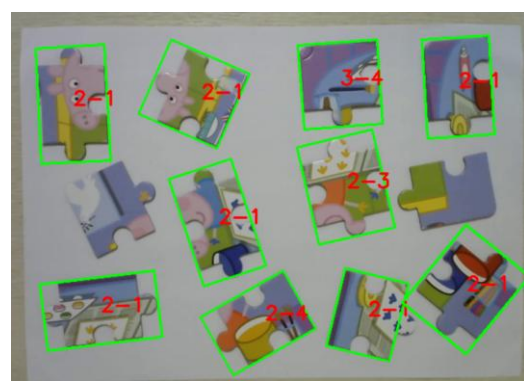
使用 data 資料夾中的照片並擷取各拼圖碎片，以更換樣板資料，如圖十所示，更改後辨識結果圖十一可見 1-4 的辨識成功率上升，然 1-1 與 2-1 的辨識成功率仍無法提升。



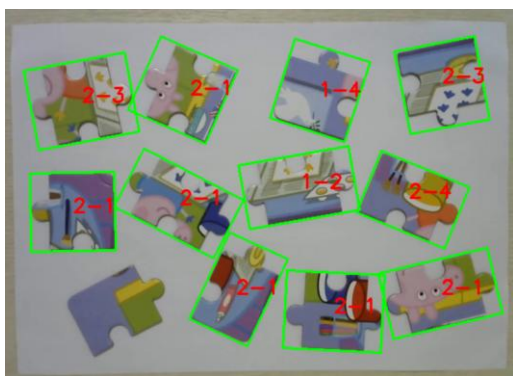
圖十、更改樣本資料



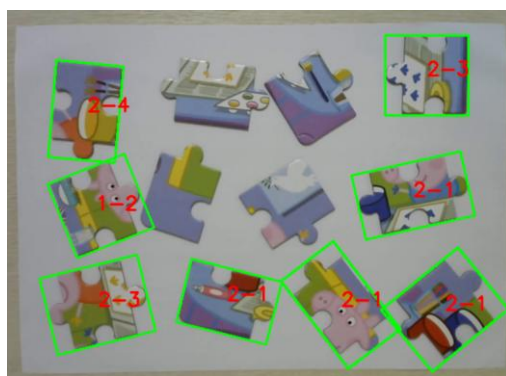
(f) 第一張



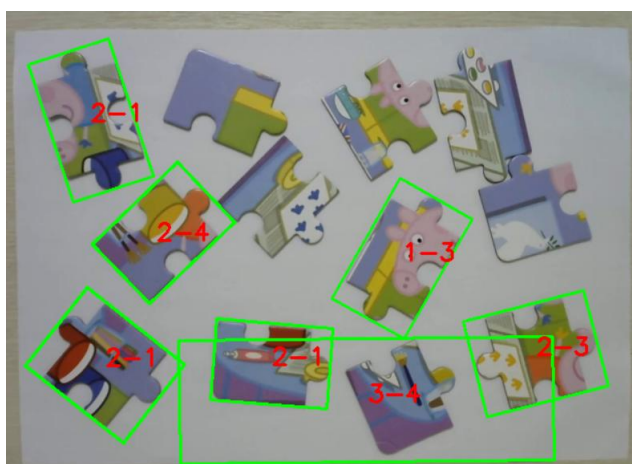
(g) 第二張



(h) 第三張



(i) 第四張



(j) 第五張

圖十一、更改後辨識結果

- 引入 YOLO 模型

引入 data 資料夾的照片進行 labeling 並產生權重，再引用 YOLOv8 模型進行訓練。

4. 題目二成果

1. 雙相機焦距計算結果：

$f_0=1200.0$

$f_0=1211.8$

2. 3D 軌跡圖繪製結果：

3D Ball Trajectory

